Copiloto de Comercios

Estado del proyecto y qué falta para el piloto en un restaurante.
Documento de traspaso — 24 de agosto de 2026.

Resumen en treinta segundos

El producto está **funcionalmente completo** para gastronomía: motor de compromisos, agente de WhatsApp, gateway, panel del local, cobro y circuito fiscal ARCA. Doce épicas cerradas y dos rondas de dogfooding con el feedback del dueño ya incorporadas. La revisión adversarial de seguridad dejó 17 hallazgos: **16 están cerrados**, el restante está documentado con su análisis y espera una decisión de diseño.

Lo que falta para que esto atienda clientes reales **no es producto**: es la infraestructura de deploy (servidor, TLS, backups), el alta en Meta para WhatsApp, y cuatro huecos concretos de código que este informe detalla. El más importante: **el agente no sabe si el local está abierto**.

Estado por componente

Componente	Stack	Estado	Tests
core	Go + Postgres	Completo	build · vet · integración y e2e verdes
agente	Python · FastMCP	Completo — 8 tools, SKILL gastro 1.3.0	51 pytest
gateway	Node · WhatsApp Cloud API	Completo	113 vitest
panel	React + Vite	Completo	139 vitest · build verde
arca-sidecar	Python · WSFEv1	Completo, contra homologación	69 pytest
landing	Vite	Publicada	—
deploy	docker compose	Sólo local: sin TLS ni backups	smoke incompleto (ver §4)

Todas las suites corren verdes al 24/08/2026. Los tests de Go necesitan Docker (testcontainers levanta un Postgres real).

1 · Cómo está armado

Cinco servicios detrás de un `docker compose`. La regla de diseño que explica casi todas las decisiones: **el núcleo es genérico y los verticales lo decoran**. El motor de compromisos no sabe qué es una hamburguesa ni un turno médico — sabe de recursos, franjas y estados. Gastronomía se sumó sin tocar el motor, y eso era exactamente la prueba de que el diseño estaba bien.

Servicio	Qué hace
core	El motor: compromisos, disponibilidad, catálogo, clientes, cobro, cierre del día. Multi-tenant: el negocio sale del token, nunca del body.
gateway	Recibe el webhook de Meta, verifica la firma, levanta el agente MCP por conversación y maneja el outbox hacia WhatsApp.
agente	Servidor MCP con las tools del asistente. Lo que el modelo puede hacer termina donde termina esa lista: no hay SQL ni negocio_id como argumento.
panel	React. El local ve el tablero de cocina, la carta, las consultas, el cierre.
arca-sidecar	Factura C contra ARCA. Aislado por tenant, con su propio lock.

El vertical activo es **gastronomía únicamente** (decisión D21, 21/08/2026). Salud está implementado y sigue verde en los tests, pero no se vende. Varios documentos y hallazgos usan "paciente" o "consultorio" en sus ejemplos por razones históricas — el bug de fondo en cada caso es genérico.

2 · Lo que se cerró en esta ronda

Cinco hallazgos de la revisión adversarial, todos con test de regresión. El detalle completo de cada uno, con el razonamiento, está en `HARDENING.md`.

H4 · El precio que se cotiza no era el que se cobraba

El agente leía `precio_base` mientras el motor congelaba el precio de la cobertura del cliente. El cliente confirmaba un número y en la caja le aparecía otro. Se cerró por las dos puntas: `GET /api/servicios?cobertura_id=` devuelve `precio_efectivo`, y el agente pide la carta con la cobertura de la ficha. Si la cobertura se declara recién al confirmar, sale un aviso para que el agente diga el número nuevo en vez de dejar que el cliente se entere pagando.

Nota de seguridad: `precio_cobertura` no tiene columna `negocio_id`. La cobertura se valida contra el negocio del token antes de usarla; sin ese chequeo, un tenant cotizaba con la lista de precios de otro.

H11 · El cierre del día equivocado cerca de medianoche

El handler resolvía el "hoy" con `time.Now()` sin conocer la zona del negocio. En el contenedor el proceso corre en UTC: a las 22:30 de Mendoza ya era el día siguiente, y el dueño abría el cierre y veía todo en cero justo cuando el local estaba trabajando. Ahora el handler pasa `nil` y el service — el único que tiene la zona — resuelve el día.

H13 · El mensaje de WhatsApp que se perdía en un deploy

Cuando un envío fallaba, el reintento vivía dentro de un `setTimeout`. Un SIGTERM en esa ventana — un deploy, un OOM — y el mensaje desaparecía sin quedar siquiera en la cola de muertos. Ahora espera en un sorted set de Redis con score igual al momento del reintento, y el worker lo barre. El test mata el proceso durante el backoff, levanta un gateway con la memoria en cero, y el mensaje igual llega.

H12 y H15

- **H12** — las ausencias del panel se construían con la zona horaria del navegador y cerraban a las 23:59, dejando el último minuto del día fuera. Ahora usan la zona del negocio y cierran a las 00:00 del día siguiente: los rangos del core son semiabiertos.
- **H15** — un cobro bonificado (total cero) le decía al dueño que le faltaba cargar el CUIT que ya tenía cargado, y lo mandaba a buscar un problema inexistente. Ahora los dos motivos se distinguen.

H17 · Abierto, esperando una decisión

La sesión del cliente viaja en el body y el core no puede verificarla. **El vector por prompt injection ya está cerrado**: el agente arma la sesión con la variable de entorno que le puso el gateway, y ninguna tool la acepta como argumento — el modelo no puede elegirla por más que se lo pidan. Lo que queda es que quien tenga el token del negocio puede mandar el teléfono de otro. Pero ese token ya es la llave del tenant, así que no agrega poder real.

Derivar la sesión del claim no se puede hoy: el token es por negocio, no por conversación. Arreglarlo de verdad significa emitir un token por conversación desde el gateway — un rediseño de auth que toca tres servicios. El análisis completo está en `HARDENING.md`; no se hizo a propósito, esperando una razón de negocio que lo justifique.

3 · Qué falta para el piloto en un restaurante

Ordenado por lo que realmente bloquea. Lo primero es trámite humano; lo segundo es código.

3.1 · Bloqueantes de infraestructura y trámite

Qué	Estado	Detalle
Servidor	Falta	Hoy el sistema corre en una notebook. Necesita un VPS.
Dominio + TLS	Falta	El webhook de Meta exige HTTPS con certificado válido. El compose publica HTTP plano y no hay reverse proxy. Falta un Caddy o Traefik con Let's Encrypt.
App en Meta	Falta	App, Business Manager verificado, número de prueba y webhook registrado.
ANTHROPIC_API_KEY	Falta	Tiene que ser una API key de plataforma. El OAuth personal no sirve: usarlo para atender clientes viola los términos de servicio.
Backups de Postgres	Falta	Pedidos reales y comprobantes fiscales sin respaldo. Un pg_dump en cron alcanza para empezar.
Plantillas de WhatsApp	**No hacen falta**	Para gastronomía el aviso de pedido confirmado sale como texto dentro de la ventana de 24 h. Se puede arrancar sin esperar ninguna aprobación.
Certificados ARCA	Opcional para testear	Con FISCAL_CIPHER_KEY vacía el módulo fiscal queda apagado: se cobra igual y no se factura.

Fecha a tener en el radar: desde el **1 de octubre de 2026** Meta cobra individualmente cada mensaje de servicio, incluidas explícitamente las respuestas generadas por IA. No bloquea el piloto, pero el costo por conversación deja de ser cero. Ver `docs/WHATSAPP.md`.

3.2 · Huecos de código

① El agente no sabe si el local está abierto — el más importante

La jornada existe: el dueño la abre y la cierra desde el panel, y el cierre del día se apoya en ella. Pero **el agente no la consulta**, y no podría aunque quisiera: `/api/jornada` está explícitamente en la lista de rutas que le devuelven 403 (`core/internal/middleware/auth.go`). El interruptor existe y no está conectado a quien atiende.

Qué pasa en la práctica: un cliente escribe a las 3 de la mañana, el bot le toma el pedido, se lo confirma, y nadie lo cocina. En un piloto real esto ocurre la primera semana.

Arreglo propuesto: sumar `GET /api/jornada/actual` a la allowlist del agente, una tool que la consulte, y una regla dura en el SKILL: con el local cerrado no se toma el pedido — se avisa el horario y se deja la consulta registrada.

② El smoke post-deploy prueba el vertical equivocado

`scripts/smoke.sh` da de alta un negocio con `"vertical":"salud"` y recorre turno → cobro → cierre. El camino que va a correr en el restaurante — pedido de varios ítems → cocina → confirmar con demora → entregado → cierre — **no se valida después de ningún deploy**. El smoke existe justamente para responder "¿puede operar ahora mismo?", y hoy lo responde sobre el vertical discontinuado.

③ docs/DEPLOY.md no existe

Está referenciado desde `docker-compose.yml`, `env.compose.example` y `EPICS.md`. Tres punteros a un archivo fantasma. Quien llegue nuevo lo va a buscar.

④ Fricción operativa que no bloquea pero cansa

- La carta se carga plato por plato desde el panel: no hay importación masiva. Para un local chico son un par de horas.
- Agregar o rotar un local significa editar `gateway/tenants.json` a mano y reiniciar el gateway: el archivo se lee una sola vez al arrancar.
- No hay monitoreo ni alertas. Si el gateway se cae un viernes a las 21, te enterás porque te llama el dueño.

4 · Orden sugerido de trabajo

Pensado para que cada paso deje algo utilizable, no para completar una lista.

#	Tarea	Por qué en ese orden
1	Jornada en el agente (§3.2 ①)	Es el único hueco que rompe la experiencia del cliente final. Acotado: allowlist, una tool y una regla en el SKILL.
2	Smoke de gastronomía (§3.2 ②)	Es lo que te va a decir si un deploy quedó sano. Sin esto, cada deploy es a ciegas.
3	VPS + dominio + TLS + backups	Sin HTTPS público no hay webhook, y sin webhook no hay WhatsApp. Los backups entran acá porque es el mismo trabajo de servidor.
4	App en Meta y número de prueba	Depende del paso 3: el webhook necesita la URL pública antes de registrarse.
5	Escribir docs/DEPLOY.md	Se escribe solo mientras hacés el paso 3, y cierra las tres referencias rotas.
6	Cargar el local piloto	Alta desde el panel admin, carta, info del local y horarios. Recién acá empieza el testeo real.

H17 y la fricción operativa del §3.2 ④ quedan fuera de esta lista a propósito. Ninguno bloquea el piloto, y resolverlos antes de tener un local usando el producto es exactamente el error que mató a los dos proyectos anteriores del equipo.

5 · Levantar el entorno

Todo el sistema, desde cero:

```
cp env.compose.example .env
# generar JWT_SECRET: openssl rand -hex 32
make up
make smoke
```

Correr las suites:

```
cd core && go test ./... # necesita Docker (testcontainers)
cd agente && uv run pytest
cd gateway && npx vitest run
cd panel && npm run build && npx vitest run
cd arca-sidecar && uv run --with-requirements requirements-dev.txt \
--with-requirements requirements-base.txt pytest
```

Gotcha del compose: `docker compose restart` no relee el `.env`. Después de tocarlo hay que hacer `docker compose up -d --force-recreate` (o `make restart`).

Dónde seguir leyendo

Archivo	Qué tiene
README.md	Puerta de entrada y estado honesto de lo que falta.
HARDENING.md	Los 17 hallazgos de seguridad, con el razonamiento de cada corrección.
EPICS.md	El plan de construcción y el estado de las doce épicas.
docs/DECISIONES.md	Las 21 decisiones de arquitectura con su porqué. Empezar por D21.

docs/ARQUITECTURA.md	Cómo se hablan los servicios.
docs/WHATSAPP.md	Costos, plantillas, onboarding de Meta y el cambio del 1/10/2026.
docs/ROADMAP.md	Las fases y, sobre todo, la regla de por qué no se construye de más.
docs/PANEL.md	Roles y qué ve cada uno.

"El núcleo se diseña genérico. El calendario libera lo que hay que vender." — docs/ROADMAP.md. Dos proyectos anteriores murieron archivados a los tres meses, no por el diseño, sino porque no había nadie esperándolos. El único bloqueante real sigue siendo el local piloto con nombre y apellido.